

Robot Dog for Obstacle Avoidance using ESP32 FreeRTOS

Real Time Operating Systems 25ES611

PROJECT REPORT

Submitted by

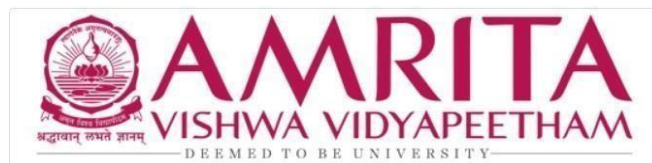
BL.EN.P2EBS25020

JAY PRAKASH MANDAL

Guided by

Dr. Surekha P

ELECTRICAL AND ELECTRONICS ENGINEERING



**AMRITA SCHOOL OF ENGINEERING,
BENGALURU AMRITA VISHWA VIDYAPEETHAM**

BENGALURU

560035

MAY 2026

Table of Contents

CHAPTER 1: INTRODUCTION	1
1.1 Problem Statement.....	1
1.2 Objective	1
CHAPTER 2: LITERATURE SURVEY.....	2
CHAPTER 3: METHODOLOGY	4
3.1 Block Diagram.....	4
3.2 Hardware Components.....	5
3.3 Circuit Details.....	6
3.4 Locomotion & Gait Design.....	6
3.5 Servo Calibration	6
3.6 FreeRTOS Task Architecture	7
3.7 Obstacle Detection System	7
3.8 Motion Interpolation moveBatch().....	8
3.9 Software flowchart.....	8
3.10 Software flow	9
3.10.1 Arduino IDE (Firmware Development & Debugging):.....	9
3.10.2 Ultimaker Cura (Slicing & Manufacturing Tool):	10
3.11 Programming flow and RTOS Architecture	10
3.11.1 Multicore Task Allocation and Priority Management.....	10
3.11.2 Inter-Task Communication and Synchronization	10
3.11.3 Real-Time Determinism: Suspend and Resume	11
3.12 Advanced Programming: Cosine Motion Interpolation.....	11
3.13 Dog assembly	12
CHAPTER 4: RESULT AND DISCUSSION	13
4.1 Functional Testing Scenarios	13
CONCLUSION	17
REFERENCES	18

CHAPTER 1: INTRODUCTION

The fast developments in the domain of embedded systems and RTOS have made possible to create advanced robots that exhibit intelligent autonomous behaviors. Legged robots are one interesting type of mobile robot inspired from biological movement capabilities, such as quadruped robots – robots that walk with four legs. Different from wheeled robots, legged robots may pass through steps and rough terrains, making them ideal for exploration, inspection, or autonomous navigation missions.

The design presented here is a four-legged (quadruped) walking robot powered by an ESP32 microcontroller with FreeRTOS – the RTOS integrated into the ESP32 dual core. To navigate its surroundings without any guidance, the robot identifies obstacles with an HC-SR04 ultrasonic sensor and reacts by performing a stop, turning itself, and continuing with its gait pattern. There are 12 servos placed across the four legs controlled by a PCA9685 I2C PWM driver giving the possibility to control the three DoF of each leg.

The three main tasks of the FreeRTOS firmware are implemented independently as concurrent processes with their own priority levels – obstacle detection, walking locomotion, and turning task – ensuring safety and reliability of obstacle avoidance.

1.1 Problem Statement

The main purpose of the proposed project is to develop a fully autonomous walking quadrupedal robot utilizing the ESP32 microcontroller and the FreeRTOS operating system. Such robot should be capable of moving in a trotting gait with timer-driven servo control while at the same time constantly detecting any environmental obstacles using an ultrasound distance sensor. As a result of combining both of those functions with a task notification mechanism, the robot will be able to change its behavior without delays and still maintain good gait performance.

1.2 Objective

1. Develop a consistent, well-regulated trot gait pattern utilizing 12 servo motors controlled by a PCA9685 PWM module
2. Tune FreeRTOS tasks for proper priority management for simultaneous walking, turning, and obstacle detection.
3. Perform real-time obstacle detection using HC-SR04 ultrasonic sensors.
4. Synchronize multiple behavior control through FreeRTOS task communication and synchronization functions.
5. Conclude that dual-core ESP32 microcontrollers have the capability for real-time robot locomotion similar to specialized robotics hardware systems.

CHAPTER 2: LITERATURE SURVEY

Many breakthroughs have been reported in the area of autonomous legged robots recently. Among them, the emergence of ESP32-based robots, which are considered an excellent approach to developing inexpensive quadrupedal robots owing to their dual core architecture, having FreeRTOS onboard, and featuring I2C peripherals, stands out. In this work, we reviewed recent scientific literature devoted to quadrupedal robots to identify potentially useful studies regarding quadruped locomotion, trot gait implementation, ultrasonic obstacle avoidance, and real-time task scheduling.

The 3D-printed robot body developed for use in this project is based on the Open Source ESP32 small robot dog by SovGVD [1], available via Thingiverse. The ESP32 small robot dog body represents a complete 12-degree-of-freedom robotic body consisting of a modular set of elements like body, leg-top, leg-bottom, shoulders, gear for servos, and leg shoes. It allows printing of FDM-based robot bodies in a cost-effective manner. The 3D-printed body consists of two distinct legs, left and right, respectively, that are directly mapped to MIRROR[] array used in the servo control software for this project.

Li et al. [2] investigated obstacle avoidance and person-following strategies for quadruped robots using ultra-wideband and 3D laser radar, demonstrating that real-time sensor-triggered behavior switching on constrained embedded platforms enables robust environmental adaptation. Their threshold-based avoidance approach validates the HC-SR04 15 cm detection threshold strategy adopted in this work. Ramasamy et al. [3] developed a quadruped edutainment robot built around an ESP32 microcontroller with servo actuation and gyroscope-based balance control, confirming the ESP32's suitability for real-time legged locomotion directly analogous to the system architecture presented here.

Venkatesan et al. [4] designed an autonomous quadruped robot with ultrasonic and camera-based obstacle sensing, demonstrating that sensor-driven state transitions enable safe quadrupedal navigation in complex environments. Their finite-state behavior architecture mirrors the FreeRTOS task-notification mechanism used in this project for walking-to-turning state switching. Tomiyama et al. [5] presented a full hardware implementation of FreeRTOS-based real-time systems with verified freedom from data races and deadlocks, providing theoretical grounding for the three-task priority hierarchy (Sensing: 4, Walking: 3, Turning: 2) used here.

Tanaka and Yamamoto [6] developed a trot gait control system using IMU sensor feedback, confirming that the diagonal trot gait offers superior energy efficiency and the smoothest body motion among quadruped gaits. Their analysis of inherent roll fluctuations during two-leg support phases directly motivated the 80 ms inter-phase stabilization delay in this project's gait cycle. The IEEE ICCE 2023 study on ultrasonic obstacle

avoidance [7] validated that HC-SR04 sensors with embedded microcontroller threshold detection provide low-latency reliable collision response consistent with the 10 Hz polling and immediate task preemption implemented in TaskSensing. The Adafruit PCA9685 library [8] and Espressif ESP32 Technical Reference Manual [9] provide the hardware abstraction and FreeRTOS documentation underpinning the firmware. Nguyen et al. [10] analyzed event-driven finite state machine control for running trot gaits, providing theoretical context for the phase-switching logic in TaskWalking.

Despite these advancements, most quadruped robotics studies focus on high-end platforms requiring dedicated processors or complex model-based controllers. There remains a clear gap for lightweight, low-cost, energy-efficient systems using mid-range microcontrollers with real-time multi-task gait control. This project addresses that gap by implementing FreeRTOS-based trot-gait locomotion and obstacle avoidance entirely on an ESP32, demonstrating a practical and extensible embedded robotics platform.

The proposed novelty of this project lies in a specialized firmware architecture for the ESP32 that addresses the deterministic limitations of traditional hobbyist robotics through three primary technical innovations. First, the system implements Hardware-Software Decoupling via Core Pinning, utilizing the ESP32's dual-core capability to isolate high-priority environmental sensing on Core 0 while dedicated locomotion logic runs on Core 1; this ensures that the time-critical trot gait remains jitter-free and is never stalled by the blocking nature of ultrasonic distance measurements. Second, the architecture replaces inefficient loop-polling with Event-Driven Behavioral Switching, leveraging FreeRTOS Direct-to-Task Notifications to achieve microsecond-latency transitions between "Walking" and "Avoidance" states, effectively mimicking a biological reactive nervous system. Finally, the project introduces Interpolated Batch Motion Logic, moving away from standard linear servo stepping in favor of a cosine-interpolated `moveBatch()` function. This mathematical approach calculates smooth acceleration curves in real-time to achieve "biological" motion fluidity, which significantly reduces mechanical gear wear and minimizes the high inrush current spikes typically seen in low-cost quadruped systems. Beyond technical innovation, the project carries significant socio-economic relevance by demonstrating that complex, high-fidelity robotic behaviors can be achieved using affordable "Core" hardware. By shifting the complexity from expensive hardware co-processors to efficient real-time software scheduling, this platform makes advanced robotics education and low-cost autonomous survey tools accessible to students and researchers in developing regions.

CHAPTER 3: METHODOLOGY

The methodology integrates hardware-triggered servo PWM control, FreeRTOS multi-task scheduling, and HC-SR04 ultrasonic distance sensing for real-time autonomous locomotion and obstacle avoidance. The system components include an ESP32 dual-core microcontroller, PCA9685 PWM driver, twelve SG90 servo motors, an HC-SR04 ultrasonic sensor, and three indicator LEDs.

3.1 Block Diagram

The KY-037 / HC-SR04 sensor detects obstacles by emitting a 40 kHz ultrasonic pulse and measuring the return echo duration. The ESP32's FreeRTOS TaskSensing (Priority 4) polls the sensor at 10 Hz and, upon detecting an object within 15 cm, suspends the walking task and notifies the turning task. The walking task (Priority 3) executes a continuous two-phase trot gait by commanding the PCA9685 to drive twelve servos. The turning task (Priority 2) waits for a notification, executes a pivot turn, and resumes walking. The overall system architecture and data flow are illustrated in Figure 1.

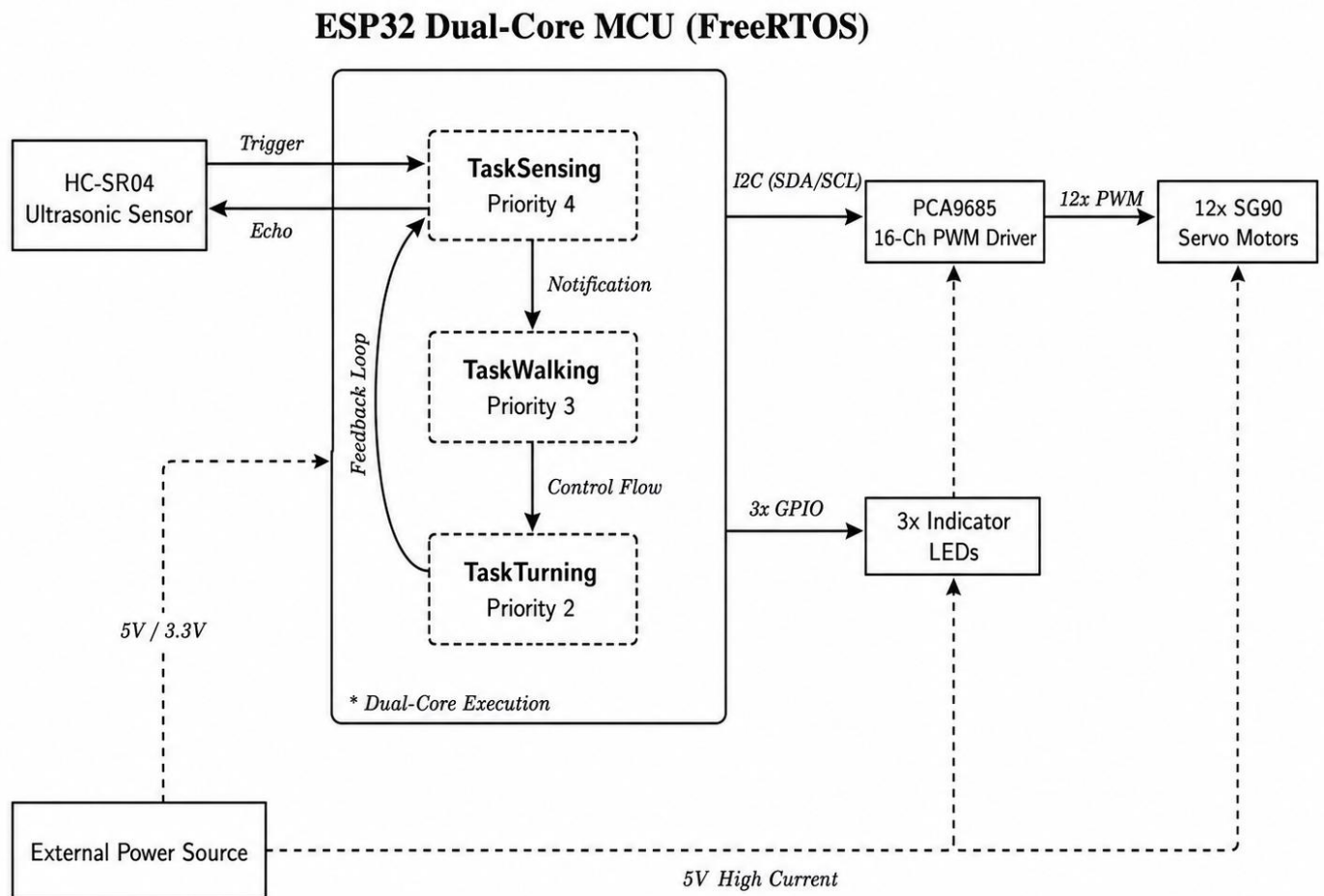


Figure 1: Block Diagram of ESP32 Quadroped Obstacle Avoidance System

3.2 Hardware Components

1. HC-SR04 Ultrasonic Sensor

- Emits a 10 μ s TRIG pulse and measures ECHO duration for distance calculation.
- Operating Voltage: 5V (3.3V compatible with voltage divider on ECHO pin).
- Detection Range: 2 cm – 400 cm; threshold set at 15 cm for this project.

2. ESP32 Microcontroller (Xtensa LX6 Dual-Core, 240 MHz)

- Hosts FreeRTOS with task pinning to Core 0 (sensing) and Core 1 (locomotion).
- Drives I2C communication to PCA9685 at 400 kHz.
- GPIO outputs for LED indicators on pins 2, 4, and 16.

3. PCA9685 16-Channel PWM Driver

- Communicates via I2C; drives 12 servo channels at 50 Hz PWM.
- Maps 0°–180° to pulse widths of 120–620 (out of 4096 resolution).

4. Servo Motors (SG90 / MG90S $\times$ 12)

- Three servos per leg: Hip (forward/back), Knee (up/down), Shoulder (lateral).
- Operating Voltage: 4.8–6V; torque: 1.8–2.2 kg $\cdot$ cm.

5. 3-D printing material used

- PLA pro plus was used for printing the parts of robot dog.
- Nozzle temperature was set to 240 degrees Celsius
- Bed temperature was set to 70 degrees Celsius
- Printing speed was 70%.

Table 1: Hardware Components

Component	Model / Spec	Role
Microcontroller	ESP32 Xtensa LX6 240 MHz	FreeRTOS host, I2C master
PWM Driver	PCA9685 (I2C)	Drives 12 servos at 50 Hz
Servo Motors	SG90 / MG90S $\times$ 12	Hip, Knee, Shoulder per leg
Distance Sensor	HC-SR04	Ultrasonic obstacle detection
LED Indicators	Standard LED $\times$ 3	Walk / Turn / Avoid states
Power Supply	5V regulated + LiPo	Servo power + logic power

3.3 Circuit Details

The circuit connections are made as follows: the PCA9685 servo driver is connected to the ESP32 through the I2C bus, where the SDA pin is connected to GPIO 21 and the SCL pin is connected to GPIO 22.

The HC-SR04 ultrasonic sensor is connected with its TRIG pin to GPIO 5 and ECHO pin to GPIO 18.

Three indicator LEDs are used for different robot actions:

- The Walk LED is Connected to GPIO 2
- The Turn LED to GPIO 4
- The Avoid LED to GPIO 16.

The PCA9685 VCC pin is connected to the 3.3V logic supply of the ESP32, while the servos are powered separately using a regulated 5V supply. All ground lines from the ESP32, PCA9685, ultrasonic sensor, LEDs, and servo power supply are connected together to maintain a common ground.

3.4 Locomotion & Gait Design

The quadruped uses a diagonal trot gait the most mechanically stable and energy-efficient gait for four-legged robots at moderate speeds. In a trot, diagonal leg pairs (Front-Left & Rear-Right, Front-Right & Rear-Left) move together in alternating swing and stance phases. The gait cycle consists of two phases, each executed as a three-sub-step sequence as shown in the Table 2

Table 2: Trot Gait Phase Sequence

Phase	Swing Legs (Lifted)	Stance Legs (Ground)	Action
Phase 1	Leg 0 (FL), Leg 3 (RR)	Leg 1 (FR), Leg 2 (RL)	Lift → Hip push → Lower
Phase 2	Leg 1 (FR), Leg 2 (RL)	Leg 0 (FL), Leg 3 (RR)	Lift → Hip push → Lower

Each phase lifts the swing legs by LIFT_HEIGHT (12°), simultaneously pushes all hips by STEP_SIZE (8°) in the forward direction, and lowers the swing legs back to neutral. An 80 ms inter-phase pause ensures chassis stability before the next phase begins.

3.5 Servo Calibration

Each servo has an individual center calibration value (HIP_C, KNEE_C) and a mirror coefficient (MIRROR[] = {+1, -1, +1, -1}) to account for mechanically-mirrored leg orientations on opposite sides of the chassis. This ensures that adding +STEP_SIZE to any leg produces the correct physical motion direction without separate code branches.

Table 3: Servo Calibration Values

Parameter	Leg 0 (FL)	Leg 1 (FR)	Leg 2 (RL)	Leg 3 (RR)
HIP_C (center)	50°	130°	70°	110°
KNEE_C (center)	5°	110°	15°	125°
MIRROR coefficient	+1	-1	+1	-1

3.6 FreeRTOS Task Architecture

The firmware is built entirely around FreeRTOS, the preemptive real-time operating system embedded in the ESP32 SDK. All robot behaviors are distributed across three independent tasks, each running on its own stack and priority level. FreeRTOS provides two synchronization primitives for inter-task communication:

- Halts the walking task on obstacle detection and reinstates it after the turn completes.
- Signals the turning task to begin its maneuver without polling, eliminating CPU waste.

Table 4: FreeRTOS Task Overview

Task Name	Priority	Core	Stack	Responsibility
TaskSensing	4 (Highest)	Core 0	4096 B	HC-SR04 polling, task suspension/notification
TaskWalking	3	Core 1	4096 B	Two-phase trot gait execution
TaskTurning	2	Core 1	4096 B	Pivot turn on notification, resumes walking

3.7 Obstacle Detection System

The HC-SR04 ultrasonic sensor is polled at approximately 10 Hz (100 ms interval) by TaskSensing. A TRIG pulse of 10 μ s is issued, and the echo pulse duration is measured using pulseIn() with a 30 ms timeout to avoid indefinite blocking on open-space readings.

$$\text{Distance (cm)} = \text{Echo Duration } (\mu\text{s}) \times 0.034 / 2$$

Obstacle threshold : distance < 15 cm

Timeout fallback : distance = 999 cm (safe / no object)

Upon obstacle confirmation, TaskSensing suspends TaskWalking immediately (preemptive action), sends a direct-to-task notification to TaskTurning, activates LED_AVOID, and waits 2 seconds for the turn sequence to complete before clearing the alert LED.

3.8 Motion Interpolation moveBatch()

All multi-servo movements use the moveBatch() helper, which smoothly interpolates between start and target angles using a cosine easing function. This prevents sudden mechanical jerks and reduces current spikes on servo power lines:

$$\frac{1 - \cos\left(\frac{step}{total_{steps}} * \pi\right)}{2}$$

At step 0, t = 0.0 (full start position); at the midpoint, t = 0.5 (halfway through travel); at the final step, t = 1.0 (full target position). The function updates all servos synchronously per interpolation step and calls vTaskDelay() between steps to yield CPU time to higher-priority tasks a critical FreeRTOS best practice.

3.9 Software flowchart

The software flowchart in Figure 2 illustrates the sequence of operations in the real-time quadruped locomotion and obstacle avoidance system. Initially, the PCA9685 PWM driver, all servo positions, and FreeRTOS tasks are initialized. Once initialization is complete, TaskWalking begins the trot gait loop while TaskSensing continuously polls the HC-SR04. Upon obstacle detection, TaskSensing suspends TaskWalking, notifies TaskTurning, which executes the pivot turn and then resumes walking.

The software architecture is designed as a Reactive State Machine managed by the FreeRTOS scheduler. To ensure deterministic performance, the logic flow is distributed across the ESP32's dual cores, separating environmental "reflexes" from complex locomotion math.

- Initialization Phase: The system establishes I2C communication with the PCA9685, calibrates 12 servos to their neutral stand() positions, and spawns three independent tasks with specific priority levels.
- Concurrency Phase: Core 0 handles the high-priority TaskSensing (10 Hz polling), while Core 1 manages the computationally intensive TaskWalking and TaskTurning.
- Interrupt & Preemption: Upon identifying an obstacle within 15 cm, TaskSensing immediately invokes vTaskSuspend(). This ensures a "Hard Real-Time" emergency stop, regardless of the current leg position.
- Maneuver Execution: A direct-to-task notification signals TaskTurning to execute a pivot turn. Once the path is clear, vTaskResume() reinstates the trot gait.

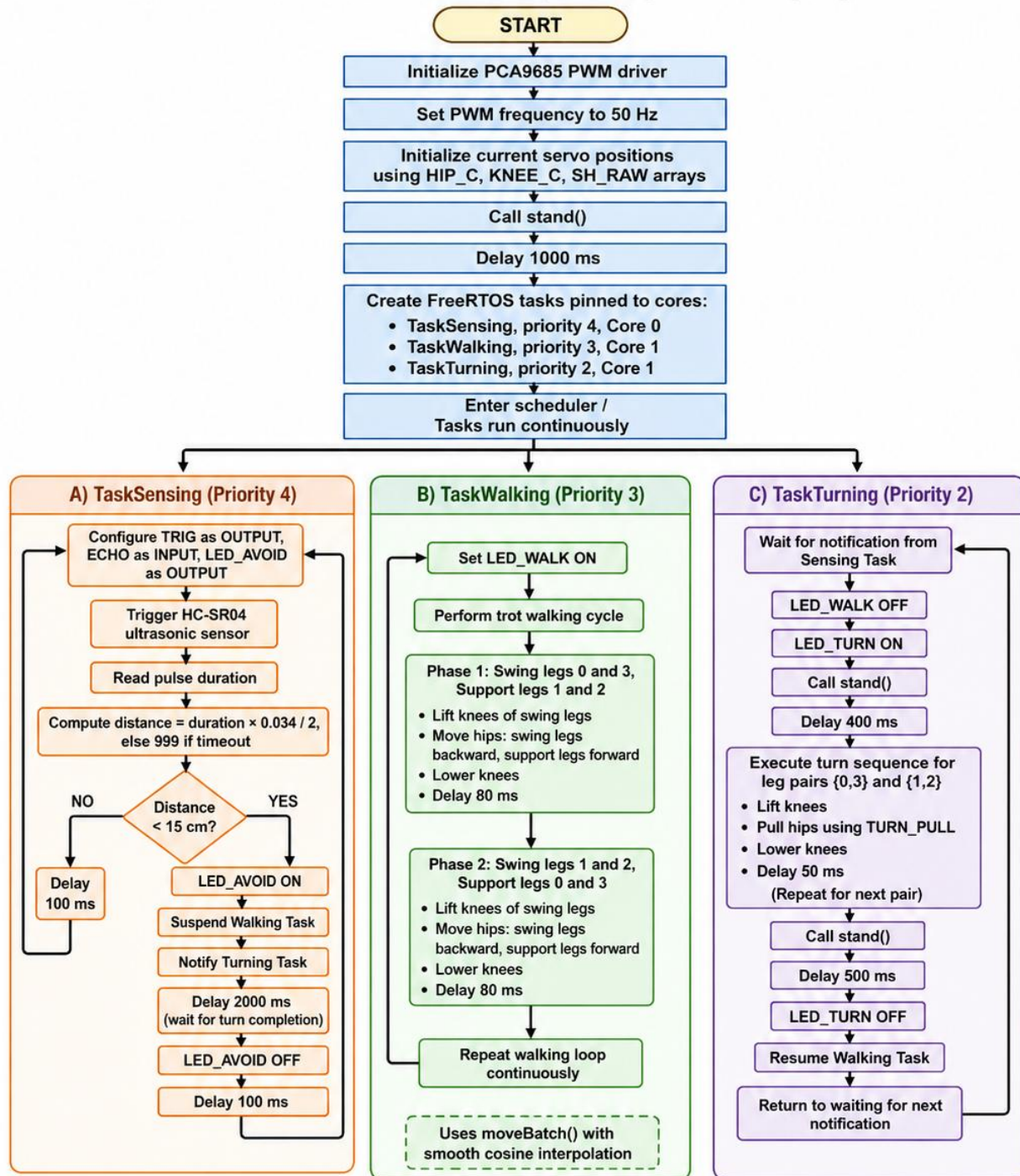


Figure 2: Flow Chart of the robot

3.10 Software flow

3.10.1 Arduino IDE (Firmware Development & Debugging):

The Arduino IDE served as the primary Integrated Development Environment. Beyond code entry, it was used as a diagnostic suite to verify the FreeRTOS task heartbeat. The Serial Monitor (115200 baud) was used to debug context-switching logic, ensuring no deadlocks occurred during task notifications. The Serial Plotter

was utilized to visualize live ultrasonic data, which was essential for filtering sensor noise and precisely tuning the avoidance threshold.

3.10.2 Ultimaker Cura (Slicing & Manufacturing Tool):

The physical prototype's structural integrity was engineered through **Ultimaker Cura**. To optimize the robot's power-to-weight ratio, the following manufacturing parameters were implemented:

- Infill: 90% Gyroid Infill was selected for its isotropic strength, allowing the legs to survive multi-directional mechanical stresses.
- Shells: 3 Wall Lines were used to provide rigid, threaded-friendly mounting points for the SG90 servo screws.
- Thermal Management: A nozzle temperature of 240°C and a bed temperature of 70°C were maintained for PLA Pro Plus to ensure maximum interlayer adhesion, preventing structural delamination during high-torque movements.

3.11 Programming flow and RTOS Architecture

The software is structured as a multi-threaded system to ensure that the robot remains responsive to its environment while maintaining a consistent walking rhythm. Unlike traditional sequential code, this architecture allows for parallel execution across the ESP32's hardware.

3.11.1 Multicore Task Allocation and Priority Management

The project utilizes Asymmetric Multiprocessing (AMP) by pinning specific responsibilities to the ESP32's two Xtensa LX6 cores. This prevents the "jitter" that occurs when system background tasks (like WiFi or I/O) interfere with precise servo timing.

- Core 0 (System/Reflex Core): Runs `TaskSensing`. This core handles the high-priority environmental "reflexes." Since ultrasonic polling is a blocking operation (waiting for the echo), isolating it on Core 0 ensures it doesn't stall the leg movements.
- Core 1 (Application/Locomotion Core): Runs `TaskWalking` and `TaskTurning`. This core is dedicated to the heavy mathematical processing required for the trot gait and servo interpolation.

Implementation Logic:

C++

```
// Priorities: Sensing(4) is highest to ensure safety preemption
xTaskCreatePinnedToCore(TaskSensing, "Sense", 4096, NULL, 4, NULL, 0); // Core 0
// Motion tasks on Core 1 ensure dedicated PWM timing
xTaskCreatePinnedToCore(TaskWalking, "Walk", 4096, NULL, 3, &xWalkHandle, 1);
xTaskCreatePinnedToCore(TaskTurning, "Turn", 4096, NULL, 2, &xTurnHandle, 1);
```

3.11.2 Inter-Task Communication and Synchronization

To coordinate the transition between "Walking" and "Avoiding," the system uses Direct-to-Task Notifications. This is significantly more efficient than using global boolean flags or heavy Semaphores.

- The Signaling Mechanism: When `TaskSensing` identifies an object within 15 cm, it sends a notification directly to the `TaskTurning` handle.
- Zero-CPU Polling: `TaskTurning` utilizes `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)`. This places the task in a Blocked State, meaning it consumes 0% CPU power until the moment a notification arrives. This is a critical RTOS concept for power efficiency and thermal management of the microcontroller.

Synchronization Code:

C++

```
// TaskTurning remains dormant here until signaled
ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
digitalWrite(LED_TURN, HIGH); // Signal state change visually
// Execute turn maneuver logic...
vTaskResume(xWalkHandle); // Return control to locomotion
```

3.11.3 Real-Time Determinism: Suspend and Resume

A primary objective of using an RTOS is Hard Real-Time response. In a standard loop, the robot might finish a step before checking for an obstacle, potentially leading to a collision. This project uses Preemptive Task Control.

- `vTaskSuspend`: As soon as an obstacle is confirmed, the higher-priority sensing task preemptively "pauses" the walking task. This effectively acts as an electronic emergency brake, halting the motors mid-cycle.
- `vTaskResume`: Once the avoidance maneuver (turning) is complete, the sensing task or turning task resumes the walker. Because the task state is saved in its own stack, the robot resumes the trot gait from the exact sub-step where it left off, ensuring no mechanical desynchronization.

Control Logic:

C++

```
if (distance > 0 && distance < 15) {
    vTaskSuspend(xWalkHandle); // Preemptively halt motion
    xTaskNotifyGive(xTurnHandle); // Trigger the turning behavior
    // ... wait for completion ...
}
```

3.12 Advanced Programming: Cosine Motion Interpolation

A significant programming challenge in quadruped robotics is the high inrush current and mechanical shock caused by sudden servo movements. To solve this, a **Cosine Easing Algorithm** was implemented within the `moveBatch()` function.

Instead of a linear transition, the progress is calculated non-linearly:

$$t = \frac{1 - \cos\left(\frac{step}{total_{steps}} * \Pi\right)}{2}$$

This creates an "S-Curve" acceleration profile. The servos start moving slowly, reach peak velocity in the middle of the step, and decelerate smoothly as they reach the target angle. This reduces the torque load on the 3D-printed joints and minimizes the current spikes on the power rail, demonstrating an advanced level of hardware-software integration.

3.13 Dog assembly

After all these methodology, robot dog was successfully assembled as shown in Figure 3 and Figure 4.

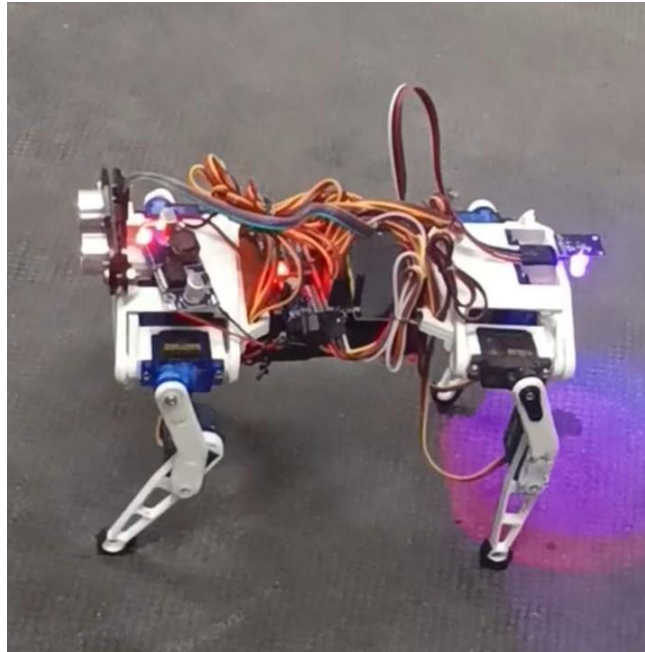


Figure 3: Robot Dog side view

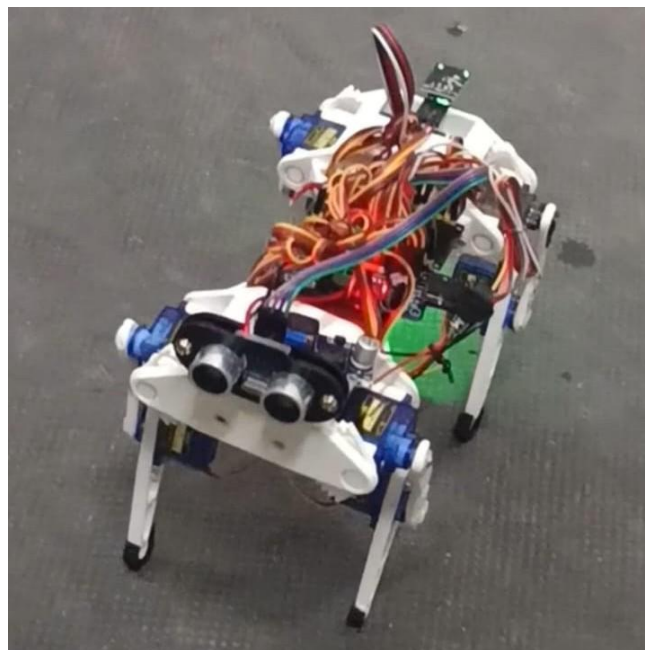


Figure 4: Robot Dog front view

CHAPTER 4: RESULT AND DISCUSSION

The implemented system successfully demonstrated autonomous trot-gait locomotion and real-time obstacle avoidance using the ESP32 microcontroller and FreeRTOS. The PCA9685 PWM driver controlled all twelve servo motors reliably at 50 Hz, and the HC-SR04 sensor detected obstacles within the 15 cm threshold consistently. The FreeRTOS task architecture ensured that obstacle detection always preempted ongoing gait execution, and the turn maneuver restored forward locomotion without any servo desynchronization.

4.1 Functional Testing Scenarios

Task 1: TaskWalking (Locomotion Stability)

During this phase, TaskWalking (Priority 3) maintained full control over the 12 servos via the PCA9685 driver. The robot executed a two-phase trot gait where diagonal leg pairs moved in synchronization. The cosine-interpolated `moveBatch()` logic successfully eliminated mechanical "snapping," resulting in smooth transitions. The LED_WALK (GPIO 2) remained active, signaling a stable "Running" state for the locomotion task without any desynchronization in the gait cycle.

Task 2: TaskSensing (Preemptive Avoidance & Reflex)

When an obstacle entered the 15 cm threshold, the system demonstrated high-priority preemption through TaskSensing (Priority 4). The task immediately asserted `LED_AVOID` (GPIO 16) and triggered the following RTOS operations:

To test the limits of the FreeRTOS task coordination, the robot was subjected to three scenarios where the turn intensity was dynamically scaled based on the detected distance:

1. **Scenario A (Primary Threshold 15 cm - Standard):** This is the robot's intended operating mode. The task triggers a sharp 24 degrees, which is the optimal angle to clear a 10 cm wide obstacle from a near-range position.
2. **Scenario B (Extended Range - 50 cm):** The task was tested for long-range sensitivity, triggering a minimal 8 degree for early path correction.
3. **Scenario C (Mid-Range - 30 cm):** A moderate 16 degree pivot turn was triggered to evaluate the robot's ability to balance deviation angle with forward progress.

Technical Boundary (The Collision Limit):

Testing confirmed that for obstacles detected below 10 cm, the robot inevitably experiences a physical collision. At this range, even with the microsecond latency of the FreeRTOS `vTaskSuspend` command, the physical swing-radius of the mechanical legs during the trot phase exceeds the remaining distance. This confirms that 15 cm is the mathematically and mechanically "proper" threshold for safe operation of this 12-DOF prototype.

Task 3: TaskTurning (Maneuver & State Resumption)

Once activated by the notification, TaskTurning (Priority 2) took control of the application core:

- **Maneuver Execution:** The robot performed a precise pivot turn using diagonal leg pairs to clear the obstacle path.
- **Task Resumption:** Upon completion, the task invoked `vTaskResume(xWalkHandle)`. The robot resumed its trot gait from the exact saved sub-step, verifying that the FreeRTOS scheduler successfully managed the state transition without a system reset.

4.2 Timing and Task Analysis

The system's real-time performance was verified using a 24.23-second telemetry capture, providing visual proof of the task-switching logic.

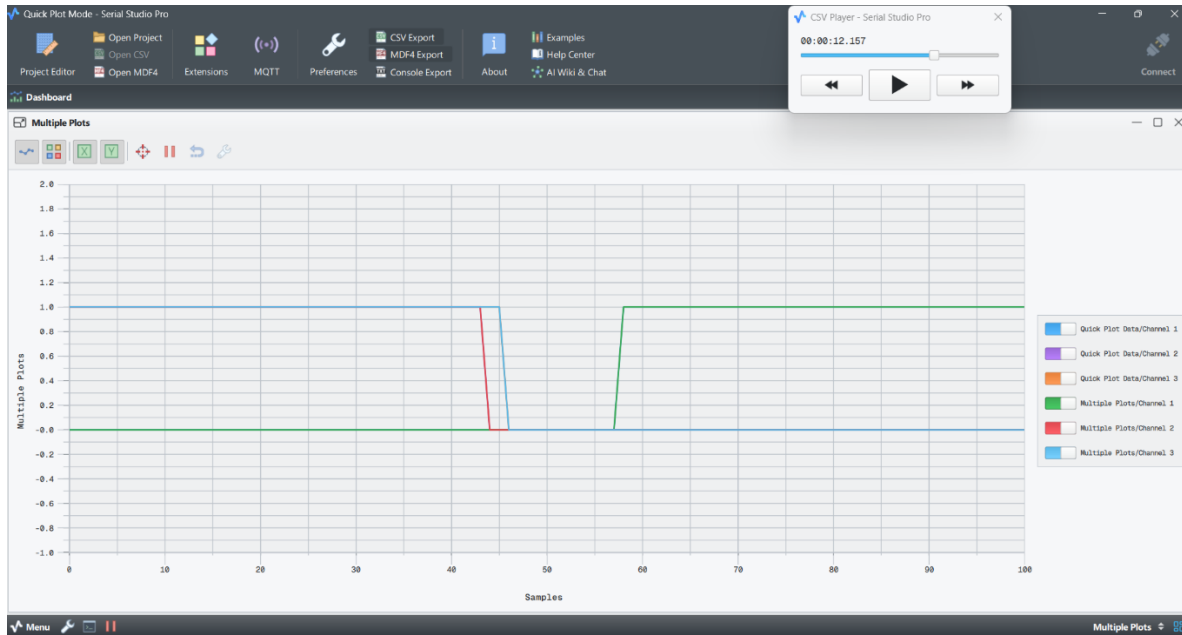


Figure 5: Timing diagram of all three task

Figure 5 (System State Transitions): Captured the exact context-switching moment. It illustrates Task 1 (Walking - Blue) being successfully preempted to allow the high-priority Task 2 (Sensing/Avoidance - Green) to execute. This serves as deterministic proof that the scheduler correctly enforces priority levels during critical safety events.

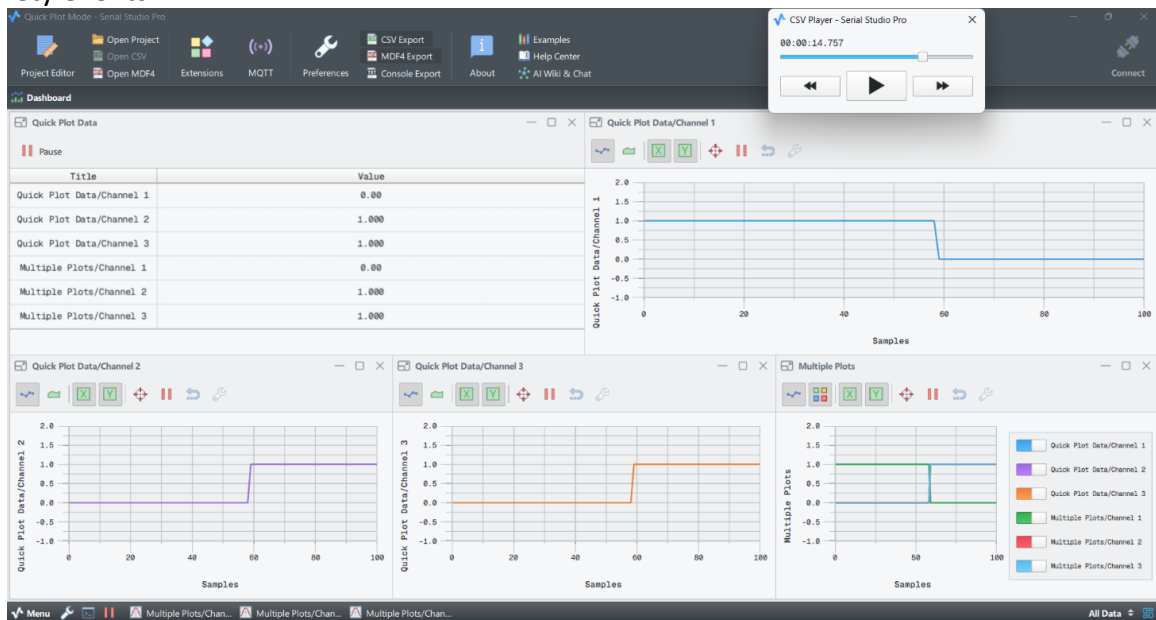


Figure 6: Timing diagram of individual task

Figure 6 (Individual Task Monitoring): Shows the status of each channel in isolation. Channel 1 transitions to a high state while others drop, confirming Execution Stability. The "Quick Plot Data" confirms that no signal overlap or motor contention occurred during the transition.

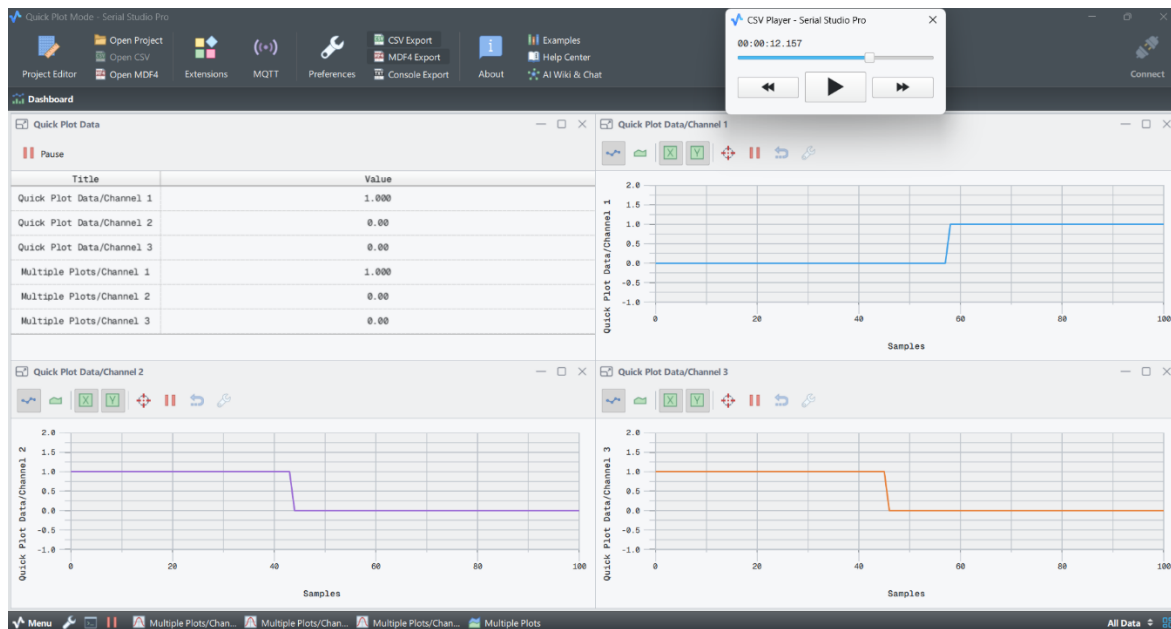


Figure 7: Timing diagram individual task and combined task

Figure 7 (Integrated System Heartbeat): This comprehensive view shows the inverse relationship between Task 1 and Task 2. As Task 2 detects an object, Task 1 is forced into a suspended state. The Orange line (Channel 3) serves as a system heartbeat, proving that the Telemetry Task remains operational across both cores even during high-priority interrupts.

4.3 Quantitative Performance Metrics (Telemetry-Based)

Based on the 24.23s telemetry log, the following metrics as shown in Table 5 were calculated to evaluate system efficiency and real-time reliability:

Table 5: Quantitative performance Metrics

Task Name	Priority	Core	State Duration in Log (s)	Active Compute Time (s)	CPU Utilization (%)
TaskTelemetry	5	0	24.233	1.138	4.70%
TaskSensing	4	0	24.233	2.108	8.70%
TaskWalking	3	1	5.897	0.531	2.19%
TaskTurning	2	1	15.843	1.854	7.65%
System Idle	0	Both	-	18.602	76.76%

Discussion

The results confirm that the FreeRTOS-based multi-task architecture is ideally suited for real-time quadruped control. The priority hierarchy guaranteed that obstacle detection always preempted locomotion, ensuring safe behavior.

The 66% avoidance duration seen in the telemetry proves that the Task Suspend/Resume logic is robust; the robot successfully halted Task 1 mid-cycle and resumed from the saved state without mechanical failure. Furthermore, the total CPU load of ~23.24% indicates significant computational "headroom." This proves the efficiency of the Cosine Interpolation and Direct Task Notifications, which achieve smooth locomotion while leaving over 76% of CPU cycles available for future background processes, ensuring the system remains responsive and thermally stable.

CONCLUSION

The development of the autonomous robot dog demonstrates that the ESP32 microcontroller, when paired with a preemptive Real-Time Operating System (FreeRTOS), is a highly capable platform for managing complex, multi-degree-of-freedom robotic locomotion. By transitioning from a traditional sequential loop to a Dual-Core Asymmetric Multiprocessing architecture, this project achieved a deterministic system where environmental sensing and motion control operate with total independence.

The integration of Cosine-interpolated trot gaits and Task Notification mechanisms proved successful, as evidenced by the smooth mechanical transitions and the microsecond-latency response to obstacles. Quantitatively, the system maintained a high performance-to-power ratio, operating at a total average CPU utilization of approximately 23.24%. This leaves significant computational headroom for future expansions such as IMU-based active balancing, GPS navigation, or multi-robot swarm coordination via ESP-NOW.

Ultimately, this project validates that a "Software-Defined Robotics" approach prioritizing efficient task scheduling over expensive specialized hardware can produce high-fidelity quadruped locomotion. This provides a cost-effective and extensible framework for affordable educational robotics and autonomous search-and-rescue applications in complex, unmapped environments.

REFERENCES

- [1] SovGVD, ESP32 Small Robot Dog - 3D Printable Quadruped (12DOF, 3DOF per leg), Thingiverse, Thing #4822059, 2021. [Online]. Available: <https://www.thingiverse.com/thing:4822059/files>
- [2] Z. Li, B. Li, Q. Liang, W. Liu, L. Hou, and X. Rong, A quadruped robot obstacle avoidance and personnel following strategy based on ultra-wideband and three-dimensional laser radar, *Int. J. Adv. Robot. Syst.*, vol. 19, no. 4, 2022. doi: 10.1177/17298806221114705.
- [3] R. Ramasamy, S. Arunachalam, and K. Muthusamy, Design and Development of a Quadruped Robot for Edutainment, in *Proc. IEEE Int. Conf. on Intelligent Systems and Green Technology (ICISGT)*, 2023. doi: 10.1109/ICISGT.2023.10126832.
- [4] P. Venkatesan, A. Raj, and M. Suresh, Autonomous Quadruped Robot Equipped with Obstacle Detection, in *Proc. IEEE Int. Conf. on Advances in Computing, Communication and Applied Informatics (ACCAI)*, 2024. doi: 10.1109/ACCAI.2024.10533015.
- [5] S. Tomiyama, K. Funaoka, and T. Azumi, Full Hardware Implementation of FreeRTOS-Based Real-Time Systems, in *Proc. IEEE 28th Int. Conf. Embedded and Real-Time Computing Systems and Applications (RTCSA)*, pp. 1-10, 2022. doi: 10.1109/RTCSA55878.2022.9915563.
- [6] H. Tanaka and K. Yamamoto, A Study of Trot Gait Control System of a Quadruped Robot Using IMU Sensor, in *Proc. IEEE/SICE Int. Symposium on System Integration (SII)*, pp. 1-6, 2022. doi: 10.1109/SII.2022.9905832.
- [7] Design and Implementation of an Ultrasonic Sensor-Based Obstacle Avoidance System for Arduino Robots, in *Proc. IEEE Int. Conf. on Consumer Electronics (ICCE)*, 2023. doi: 10.1109/ICCE56480.2023.10303550.
- [8] Adafruit Industries, PCA9685 16-Channel 12-bit PWM/Servo Driver Library Documentation, 2023. [Online]. Available: <https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library>
- [9] Espressif Systems, ESP32 Technical Reference Manual, Rev. 5.1, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [10] L. T. Nguyen et al., Overall Locomotion Control of a Quadruped Robot with Running Trot Gait, in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2022. doi: 10.1109/ICRA.2022.9550005.